Jdbc и Hibernate

hh team

Обо мне

Выпускник школы программистов hh
2021-2022 гг.

Бэкенд-разработчик в команде A-Shot

План лекции

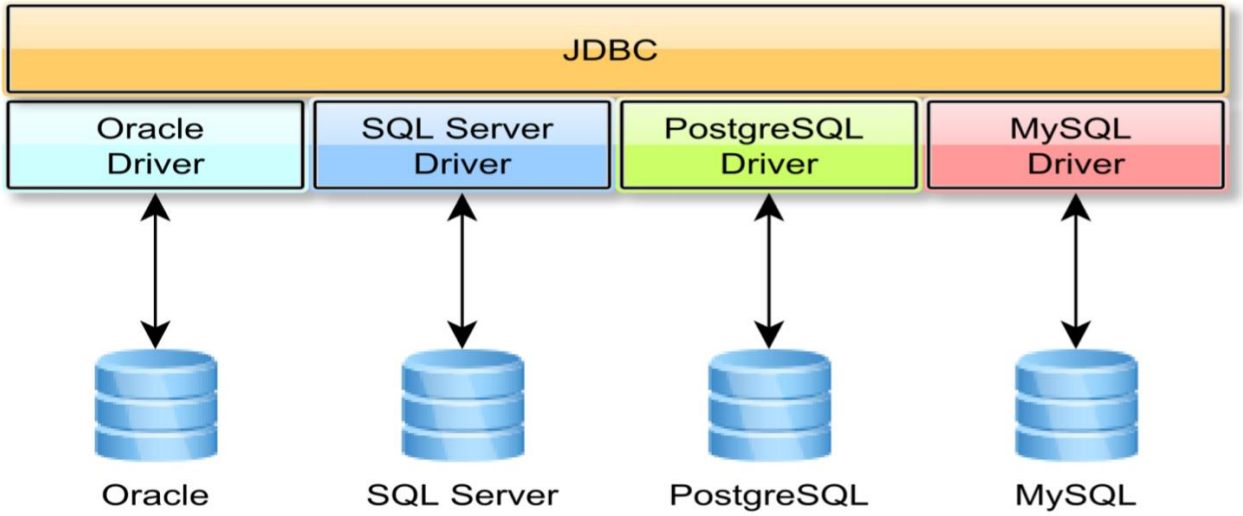
1) JDBC
   - Немного теории
   - Примеры кода

Перерыв на 10 мин

2) Hibernate
   - Немного теории
   - Примеры кода

JDBC

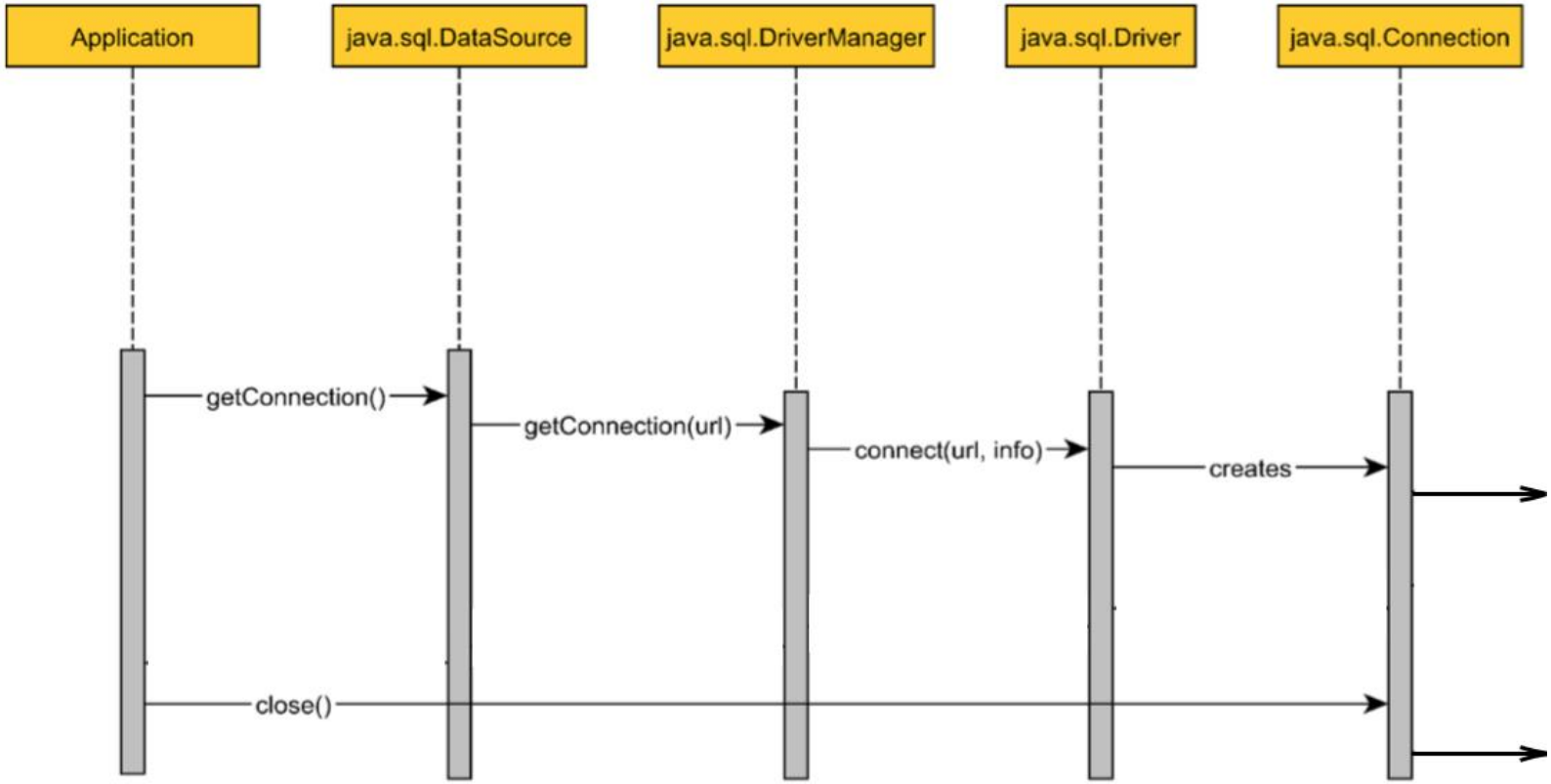
JDBC (Java DataBase Connectivity) - это API для работы с базами данных из java приложений. Определяет общий интерфейс для взаимодействия программ с базами данных.



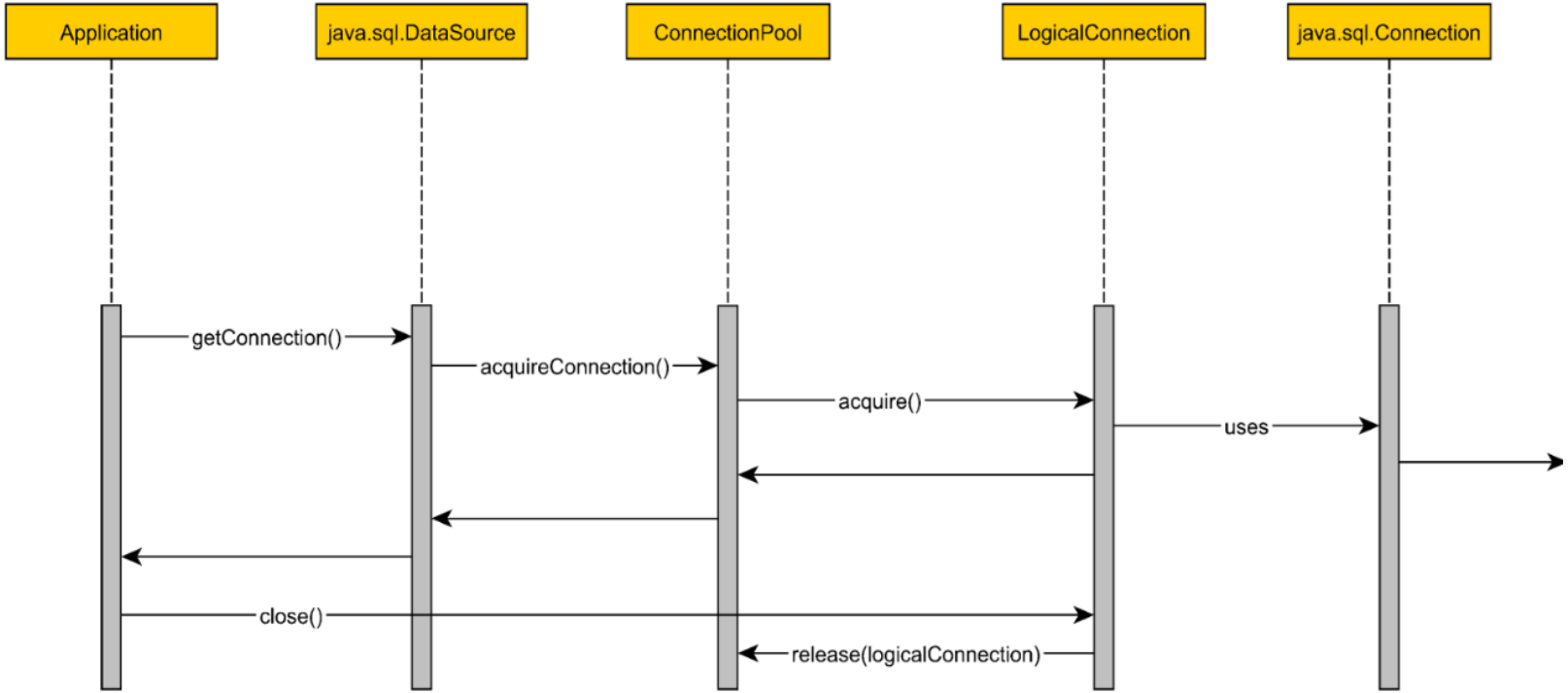
Получение соединения

- Driver
  - Предоставляет физическое соединение
  - Для каждой СУБД свой
- DriverManager
  - В составе JDBC
  - Упрощает получение соединения
  - getConnection(String url)
- DataSource
  - Абстракция более высокого уровня над DriverManager

Получение соединения. Простой вариант

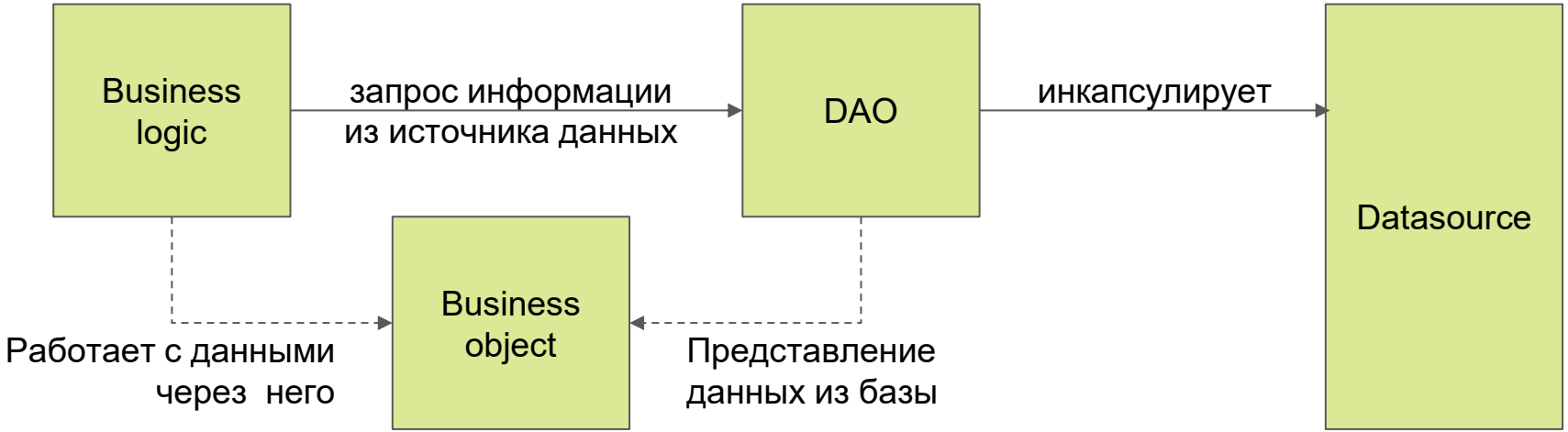


Получение соединения с пулом соединений



Шаблон Data Access Object

Используйте Data Access Object (DAO) для абстрагирования и инкапсуляции доступа к источникам данных.



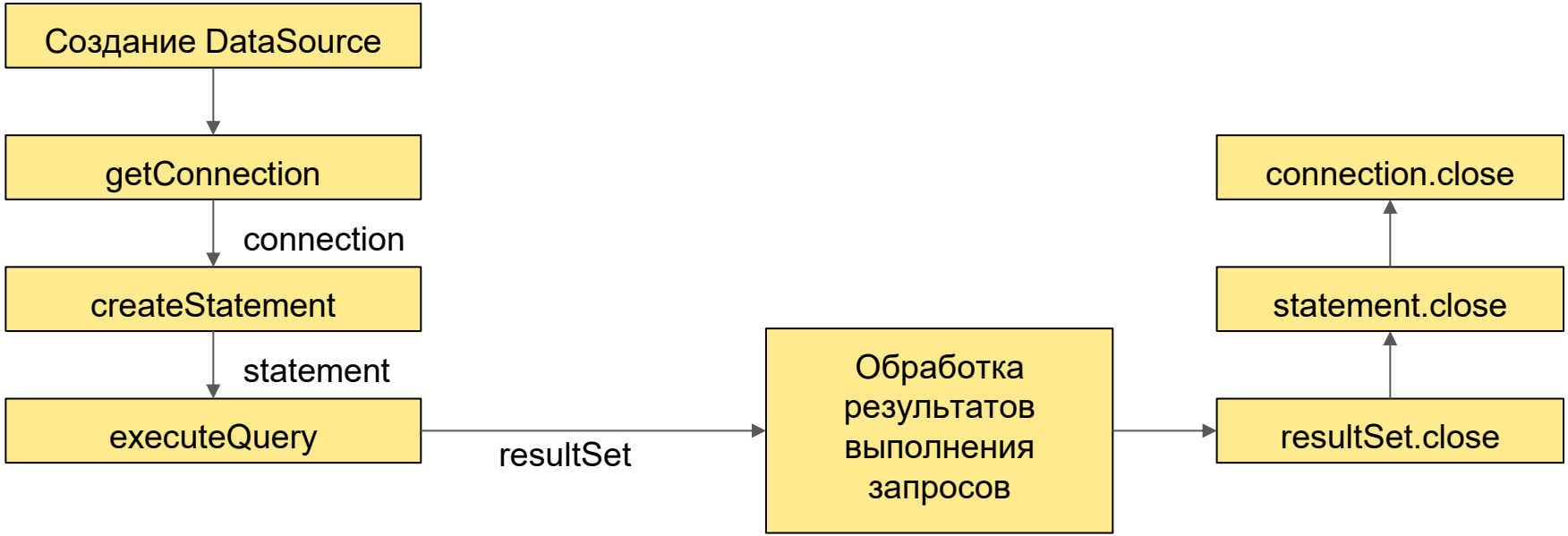
Интерфейс Statement

Statement - интерфейс, через который выполняются SQL запросы.
Два основных метода executeUpdate() и executeQuery()

- Statement
  - connection.createStatement()
  - для запросов без параметров
  - можно выполнить несколько запросов
- PreparedStatement (extends Statement)
  - connection.prepareStatement(sqlQuery)
  - для повторяющихся запросов
  - для запросов с параметрами
  - связан только с одним запросом

CallableStatement

Порядок вызовов



Итоги

Создали простой DataSource

Использовали Statement и PreparedStatement

Использовали executeUpdate() и executeQuery()

Посмотрели как обрабатывать результаты

Не забываем закрывать ресурсы

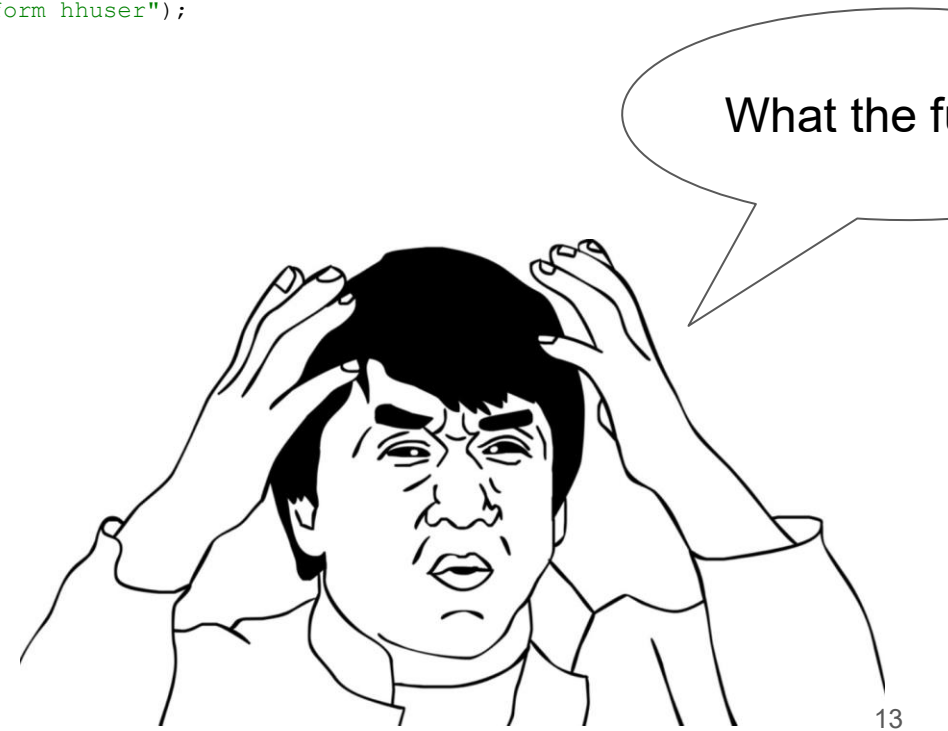
CallableStatement

Что не так?

- Try, try, try
- Checked Exceptions
- Позиционные аргументы в PreparedStatement (Да и еще с 1 !!!)
- Для новой сущности придется повторить код
- Сложно реализовать связанные сущности (один-ко-многим, многие-ко-многим и др.)

Что бы мы делали без **try-with-resources**

```
public Set<User> getAll() {
 Connection connection = null;
 try {
   connection = dataSource.getConnection();
   Statement statement = null;
   try {
     statement = connection.createStatement();
     ResultSet resultSet = null;
     try {
       resultSet = statement.executeQuery("select * form hhuser");
       // НАКОНЕЦ-ТО ОБРАБОТКА
     } catch (SQLException e) {
         e.printStackTrace();
     } finally {
       if (resultSet != null) {
         resultSet.close();
       }
     }
   } catch (SQLException e) {
     e.printStackTrace();
   } finally {
     if (statement != null) {
       statement.close();
     }
   }
 } catch (SQLException e) {
   e.printStackTrace();
 } finally {
   if (connection != null) {
     try {
       connection.close();
     } catch (SQLException e) {
```



Prepared statement

```
try (Connection connection = dataSource.getConnection();
    PreparedStatement preparedStatement = connection.prepareStatement(
        "insert into hhuser (first_name, last_name) values (?, ?)",
        Statement.RETURN_GENERATED_KEYS)
) {
   preparedStatement.setString(1, user.getFirstName());
   preparedStatement.setString(2, user.getLastName());
   preparedStatement.executeUpdate();

   try (ResultSet generatedKeys = preparedStatement.getGeneratedKeys()) {
     if (generatedKeys.next()) {
       user.setId(generatedKeys.getInt("user_id"));
     }
   }
}
```

ВОПРОСЫ

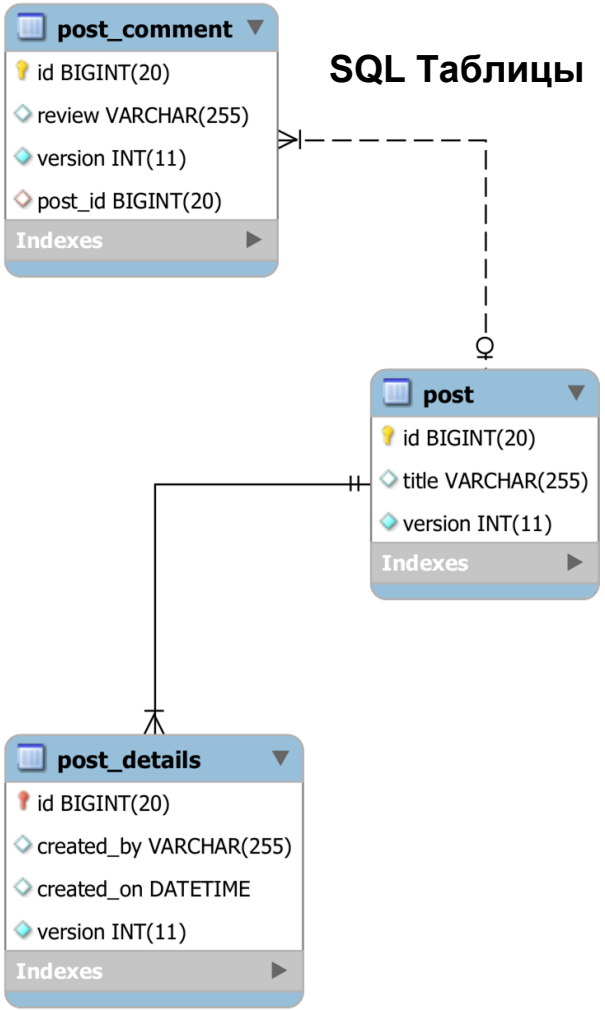
Hibernate

Hibernate was the most successful Java ORM implementation.

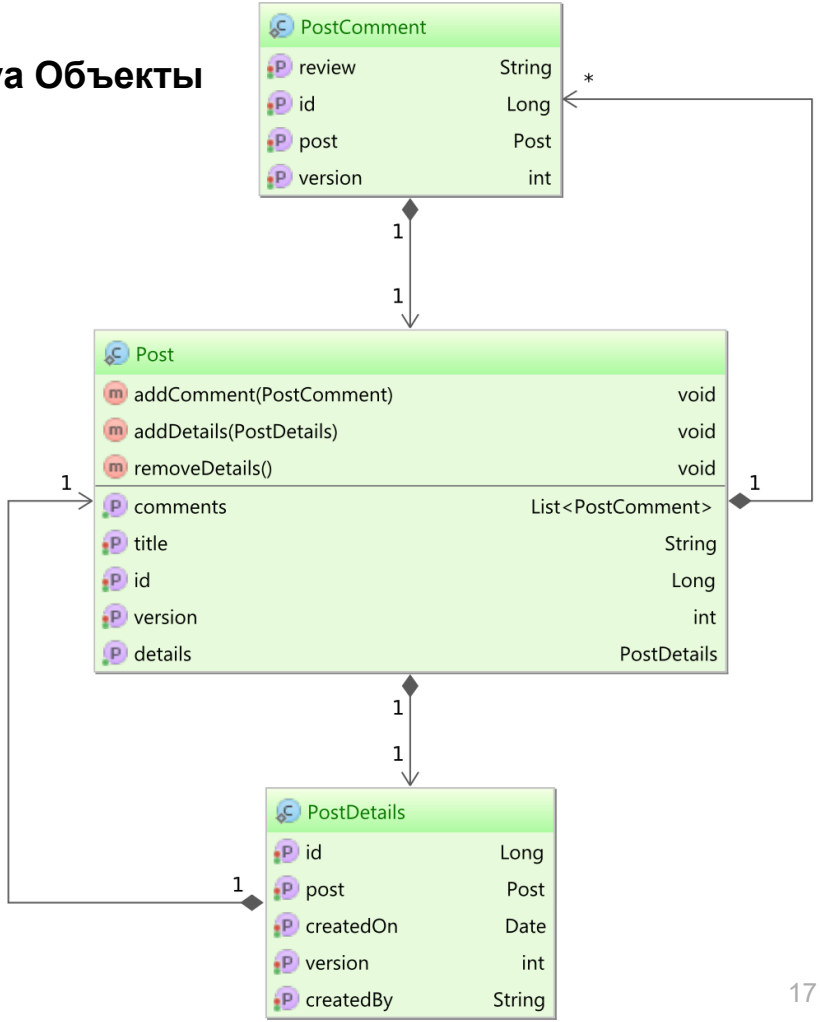
by baeldung

- Инструмент ORM (Object-Relational Mapping)
- Помогает преодолеть несоответствие между миром реляционных БД и объектной моделью
- Автоматически транслирует изменение состояния объектов в SQL запросы

SQL Таблицы



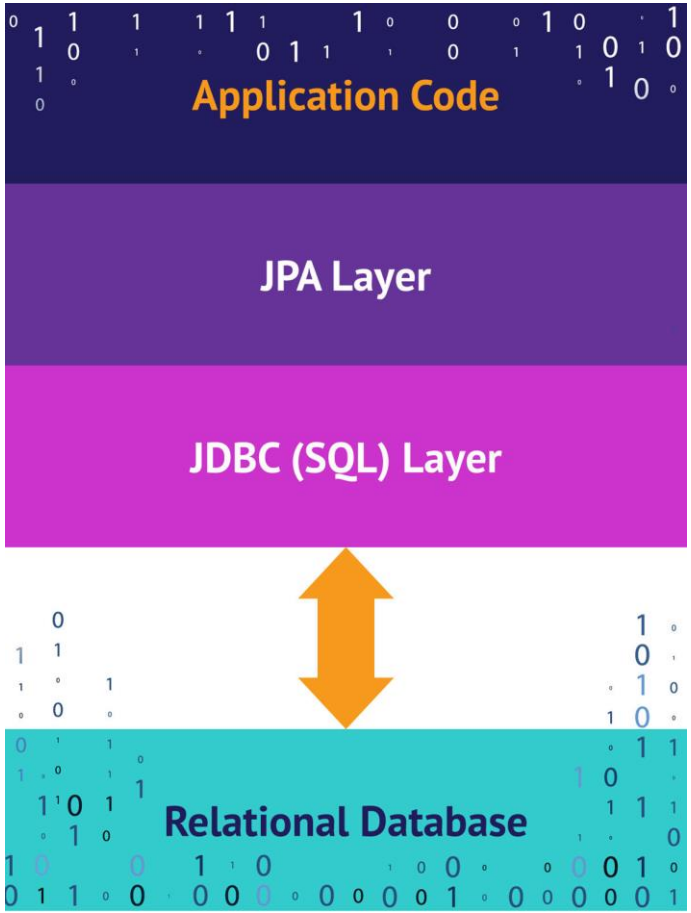
Java Объекты



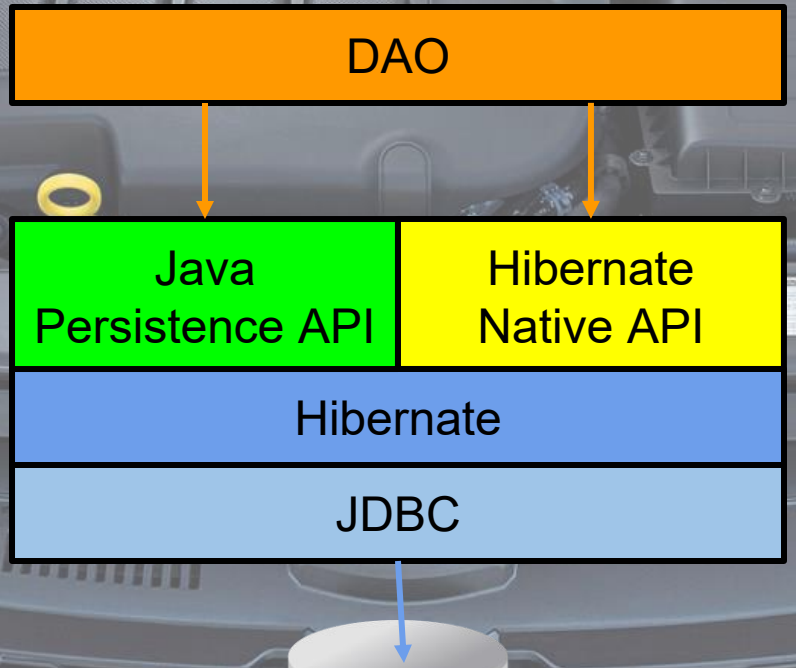
Коротко о JPA

JPA (Java Persistence API) - это спецификация, интерфейс. Часть Java EE.

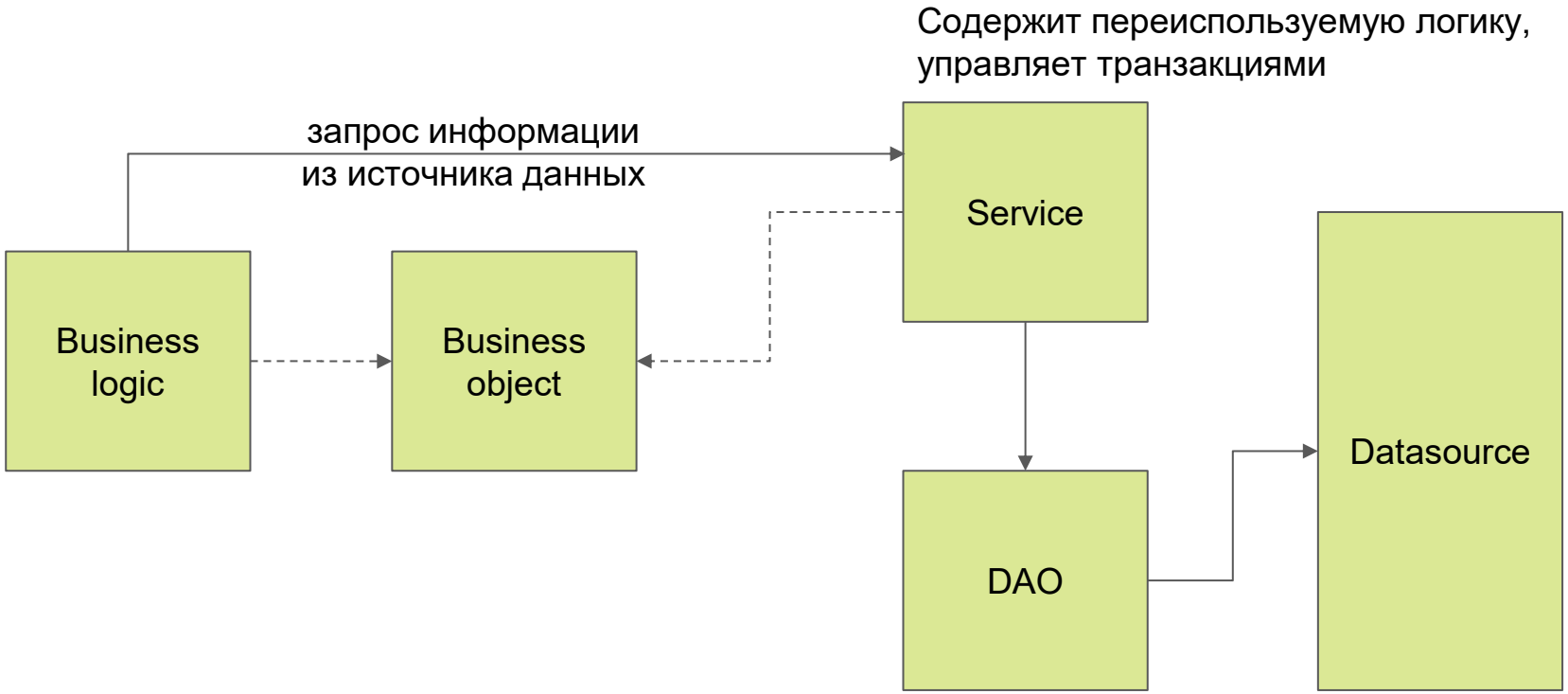
Hibernate - реализует JPA.



Под капотом


DAO
Java Persistence API
Hibernate Native API
Hibernate
JDBC

Обновляем схему приложения



Требования для Entity

- @Entity
- Конструктор без аргументов
- Non final class
- Get-ы и set-ы (если поля приватные)

- Поле id
- @Table, если имя таблицы отличается от имени класса
- @Column, если имена полей класса отличается от полей в базе

```
@Entity
@Table(name = "hhuser")
public class User {
    @Id
    @GeneratedValue(strategy =
            GenerationType.IDENTITY)
    @Column(name = "user_id")
    private Integer id;

    @Column(name = "first_name")
    private String firstName;

    @Column(name = "last")
    private String lastName;

    public User() {
    }
```

SessionFactory

- Создаёт экземпляры сессии
- Обычно один объект на приложение / бд
- Дорогой в создании
- Потокобезопасный
- Неизменяемый

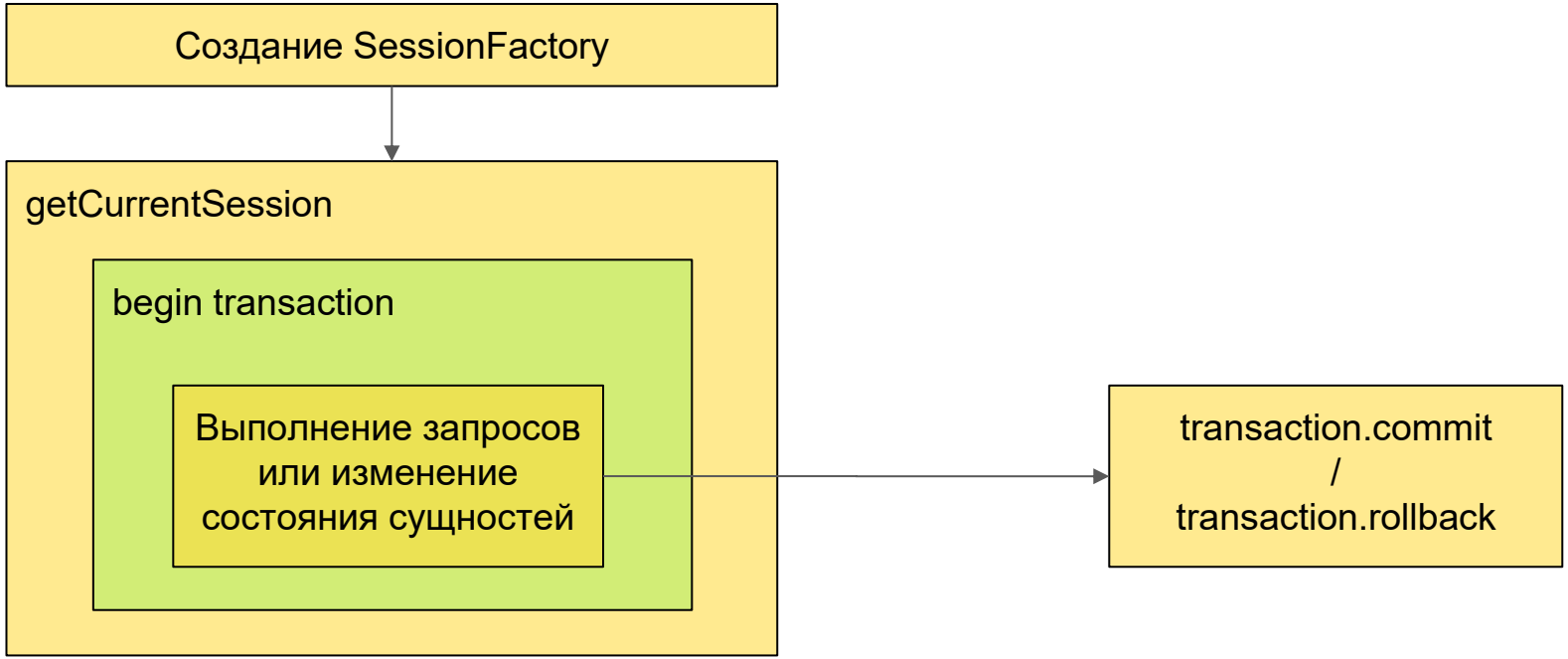
Session

- Представляет собой соединение между Java-приложением и БД
- Однопоточный
- Дешевый в создании
- Через этот объект происходит взаимодействие с БД

Transaction

- Однопоточный
- Дешевый в создании
- Для определения физических границ транзакции

Порядок вызовов



Порядок вызовов

```
Session session = sessionFactory.getCurrentSession();
Transaction transaction = session.getTransaction();
transaction.begin();
try {
  session
          .createQuery("….")
          .exexuteUpdate();
  transaction.commit();
} catch (RuntimeException e) {
  transaction.rollback();
}
```

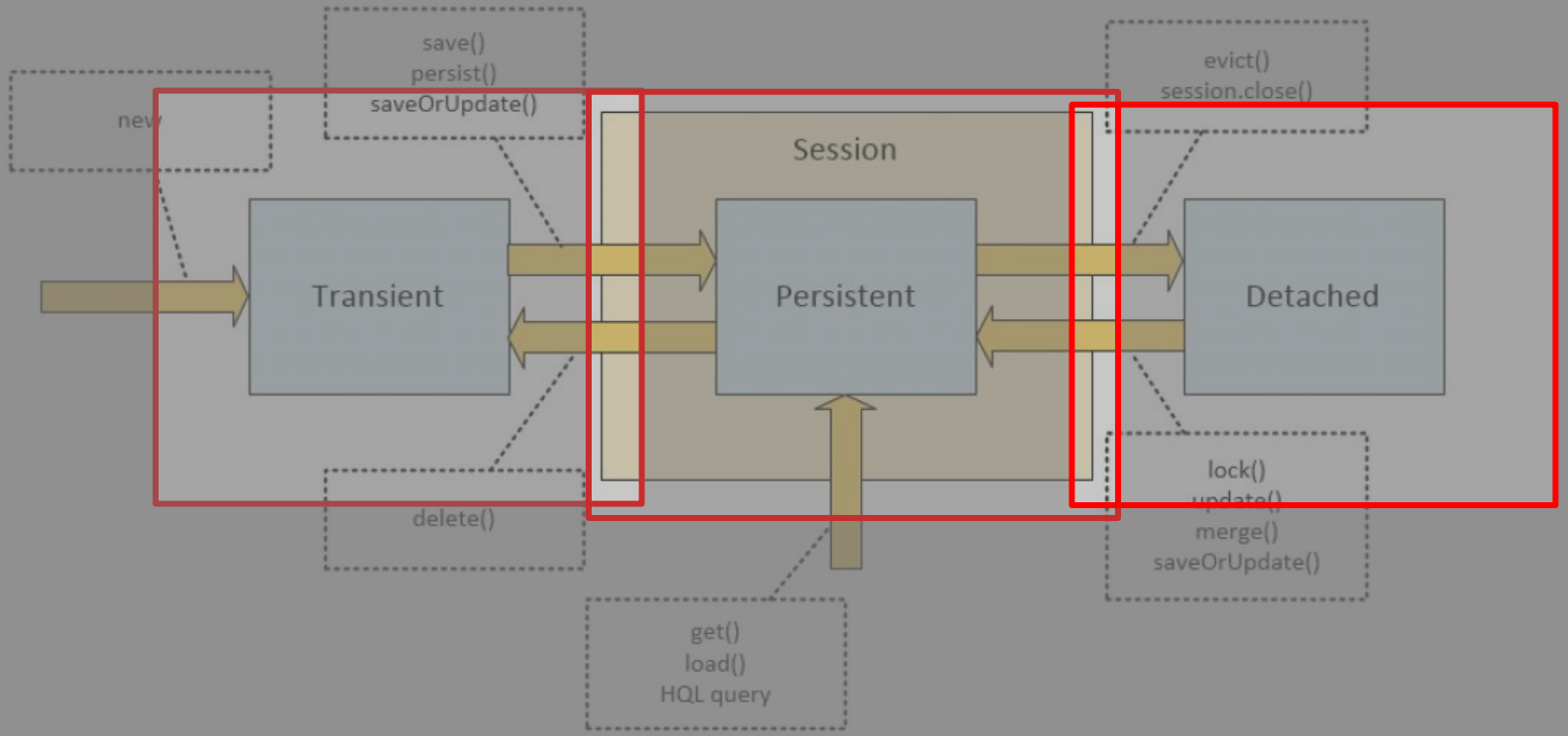
Java Persistence Query Language

SQL работает таблицами в базе данных и их столбцами

JPQL работает с объектами и их полями

Не зависит от диалекта определённой СУБД

Состояния сущности и переходы между ними.



Запросы с аргументами

```
Optional<Address> currentAddressRecord = session().createQuery(
   "SELECT a FROM Address a JOIN FETCH a.applicantAddress aa WHERE " +
       "aa.id.applicantUserId = :applicantUserId AND " +
       "a.city = :city AND " +
       "a.street = :street AND " +
       "a.building = :building AND " +
       "a.marker.lat = :lat AND " +
       "a.marker.lng = :lng", Address.class)
   .setParameter("applicantUserId", applicantUserId)
   .setParameter("city", city)
   .setParameter("street", street)
   .setParameter("building", building)
   .setParameter("lat", lat)
   .setParameter("lng", lng)
   .uniqueResultOptional();
```

Практическая часть

- Подключение зависимости
- Оформление и регистрация сущности User
- Создание SessionFactory
- Новое Dao с Hibernate
- Управление транзакциями

Оформление сущности

```
@Entity
@Table(name = "hhuser")
public class User {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   @Column(name = "user_id")
   private Integer id;

   @Column(name = "first_name")
   private String firstName;

   @Column(name = "last")
   private String lastName;

   public User() {
   }
```

Создание SessionFactory

```
private static SessionFactory createSessionFactory() {
 ServiceRegistry serviceRegistry = new StandardServiceRegistryBuilder()
   .loadProperties("hibernate.properties")
   .build();

 Metadata metadata = new MetadataSources(serviceRegistry)
   .addAnnotatedClass(User.class)
   .buildMetadata();

 return metadata.buildSessionFactory();
}
```

Управление транзакциями

```
Optional<Transaction> transaction = beginTransaction();
try {
   // выполняем необходимые операции
   transaction.ifPresent(Transaction::commit);
   // возвращаем результат
} catch (RuntimeException e) {
   transaction
           .filter(Transaction::isActive)
           .ifPresent(Transaction::rollback);
   throw e;
}
```

Получение всех сущностей

```
sessionFactory
        .getCurrentSession()
        .createQuery("from User", User.class)
        .list();
```

Создание новой записи

```
public void saveNew(User user) {
   session().persist(user);
}
```

В реальной жизни

```
@Transactional(readOnly = true)
public AvailablePublications getAvailablePublications(int employerManagerId) {

 EmployerManagerHibernateStub employerManager =
genericDao.getOrThrow(EmployerManagerHibernateStub.class, employerManagerId);

 return new AvailablePublications(
     Arrays.stream(VacancyPublicationType.values())
         .map(pt -> Map.entry(pt, PublicationsBalance.of(pt,
             balanceCalculator.getBalance(employerManager.getEmployerId(), pt),
             balanceCalculator.getQuota(employerManager.getId(), pt))))
         .filter(e -> !e.getValue().getPublications().isEmpty() ||
AVAILABLE_FOR_PURCHASE.contains(e.getKey()))
         .collect(toMap(Entry::getKey, Entry::getValue))
 );
}
```

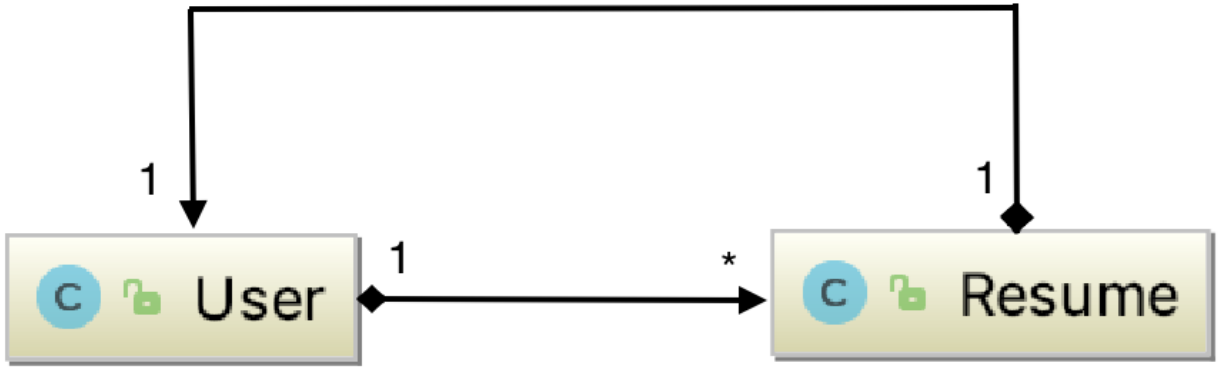
Подведем итоги

- Оформили и зарегистрировали сущности
- Познакомились с API основных элементов hibernate
- Управление транзакциями

Связи между сущностями

- `@ManyToOne`
- `@OneToMany`
- `@ManyToMany`
- `@OneToOne`
- + односторонние или двусторонние связи

Пример связи @OneToMany



user_id	first_name	last_name
1	Sarah	Connor
2	John	Connor

resume_id	title	User_id
1	gun master	1
2	mother	1
3	leader	2

Many-To-Many

employee_id	first_name	last_name
1	Sarah	Connor
2	John	Connor

employee_id	project_id
1	1
1	2
2	2

project_id	title
1	The Terminator
2	Terminator 2: Judgment Day

Many-To-Many

```
@Entity
@Table(name = "Employee")
public class Employee {

 @ManyToMany(cascade = { CascadeType.ALL })
 @JoinTable(
     name = "Employee_Project",
     joinColumns = { @JoinColumn(name = "employee_id") },
     inverseJoinColumns = { @JoinColumn(name = "project_id") }
 )
 Set<Project> projects = new HashSet<>();
}
```

Коротко о проблемах работы с hibernate

LAZY INITIALIZATION EXCEPTION

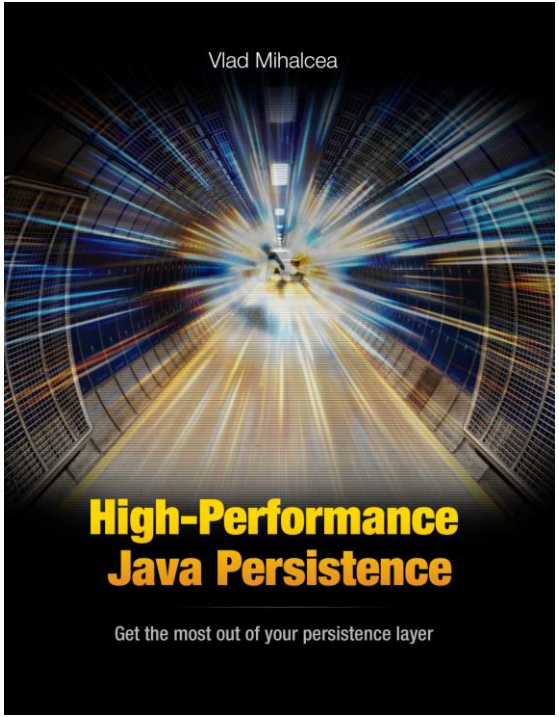
N + 1 проблема

Так же связана с lazy loading

Как выполнять запрос?

- JPQL (HQL)
- JPA Criteria
- Native sql

Полезные ссылки



https://docs.oracle.com/javase/tutorial/jdbc/basics/index.html

https://docs.jboss.org/hibernate/orm/6.6/userguide/html_single/Hibernate_User_Guide.html

https://vladmihalcea.com/how-does-hibernate-store-second-level-cache-entries/

https://vladmihalcea.com/how-does-hibernate-query-cache-work/

hh team